

BAB XII

TRANSACTION CONTROL LANGUAGE (TCL)

12.1 Tujuan Praktikum

1. Mahasiswa dapat memahami konsep dasar *Transaction Control Language* (TCL) dalam sistem basis data relasional.
2. Mahasiswa mampu menggunakan perintah `COMMIT` untuk menyimpan perubahan data secara permanen dalam *database*.
3. Mahasiswa mampu menggunakan perintah `ROLLBACK` untuk membatalkan perubahan data yang belum disimpan secara permanen.
4. Mahasiswa mampu menggunakan perintah `SAVEPOINT` untuk menetapkan titik pemulihan dalam sebuah transaksi.
5. Mahasiswa mampu menguji efek dari perintah TCL dalam berbagai skenario seperti `INSERT`, `UPDATE`, dan `DELETE` data.

12.2 Dasar Teori

Transaction Control Language (TCL) merupakan bagian dari bahasa SQL yang digunakan untuk mengelola transaksi dalam sistem basis data relasional. Dalam konteks ini, transaksi diartikan sebagai serangkaian instruksi SQL yang dijalankan sebagai satu kesatuan logis dan tidak dapat dipisahkan. Transaksi menjamin bahwa data dalam basis data tetap konsisten, bahkan ketika terjadi kegagalan sistem atau kesalahan dalam proses *input* data. Oleh karena itu, TCL sangat penting dalam menjaga integritas dan keandalan data. TCL mencakup tiga perintah utama yaitu `COMMIT`, `ROLLBACK`, dan `SAVEPOINT`, yang masing-masing memiliki peran strategis dalam mengendalikan bagaimana dan kapan perubahan data disimpan atau dibatalkan.

Perintah `COMMIT` digunakan untuk menyimpan semua perubahan yang dilakukan selama transaksi ke dalam basis data secara permanen. Setelah perintah ini dijalankan, perubahan data tidak dapat dibatalkan dan akan langsung terlihat oleh pengguna lain. Sebaliknya, perintah `ROLLBACK` digunakan untuk membatalkan perubahan yang telah dilakukan selama transaksi, baik sebagian maupun seluruhnya, jika terjadi kesalahan atau data yang dimasukkan tidak sesuai. Hal ini memungkinkan sistem untuk kembali ke keadaan stabil sebelumnya. Sedangkan

`SAVEPOINT` merupakan fitur yang memungkinkan pengguna membuat titik-titik tertentu dalam transaksi sebagai penanda. Dengan adanya `SAVEPOINT`, pengguna dapat melakukan `ROLLBACK TO` ke titik tertentu tanpa harus membatalkan seluruh transaksi. Ini sangat berguna dalam transaksi yang kompleks, di mana sebagian perubahan ingin dipertahankan dan sebagian lainnya dibatalkan.

Selain perintah TCL, transaksi dalam SQL juga erat kaitannya dengan perintah DML (*Data Manipulation Language*), yaitu `INSERT`, `UPDATE`, dan `DELETE`. Perintah `INSERT` digunakan untuk menambahkan data baru ke dalam tabel, `UPDATE` digunakan untuk mengubah atau memodifikasi data yang sudah ada, dan `DELETE` digunakan untuk menghapus data dari tabel berdasarkan kondisi tertentu. Ketiga perintah ini merupakan bagian dari proses manipulasi data yang sering kali harus dikendalikan melalui mekanisme transaksi agar tidak menyebabkan ketidakkonsistenan data, terutama dalam sistem yang digunakan oleh banyak pengguna secara bersamaan (*multi-user system*). Dalam praktiknya, perubahan yang dilakukan melalui `INSERT`, `UPDATE`, atau `DELETE` tidak akan benar-benar tersimpan secara permanen sampai perintah `COMMIT` dijalankan. Sebaliknya, jika ditemukan kesalahan, perintah `ROLLBACK` dapat digunakan untuk membatalkan seluruh perubahan.

Secara keseluruhan, pemahaman mengenai TCL dan integrasinya dengan perintah-perintah DML sangatlah penting bagi pengelolaan basis data yang aman, akurat, dan efisien. Dalam skenario nyata, terutama pada sistem informasi berskala besar seperti perbankan, akademik, atau manajemen keuangan, kemampuan untuk mengelola transaksi dengan benar menjadi fondasi utama untuk menjamin integritas dan keandalan data. Dengan menguasai perintah `COMMIT`, `ROLLBACK`, `SAVEPOINT`, serta perintah `INSERT`, `UPDATE`, dan `DELETE`, seorang pengembang atau administrator basis data dapat memastikan bahwa setiap transaksi yang dilakukan dapat dikendalikan sepenuhnya sesuai dengan kebutuhan sistem dan kebijakan data yang berlaku (Ochilboyev, 2024).

12.3 Hasil dan Pembahasan

1. *Query*

```
UPDATE copy_departments SET manager_id= 101
WHERE department_id = 60;

SAVEPOINT one;
```

Hasil:

```
SQL> UPDATE copy_departments SET manager_id= 101
2  WHERE department_id = 60;

1 row updated.

SQL>
SQL> SAVEPOINT one;

Savepoint created.

SQL>
```

Gambar 12.1 Menampilkan hasil perubahan nilai kolom `manager_id` menjadi 101 pada baris dengan `department_id` 60 di tabel `copy_departments`

Penjelasan:

Pada *query* tersebut digunakan untuk memperbarui kolom `manager_id` untuk `department_id = 60` pada tabel `copy_departments`. `SAVEPOINT one` disimpan setelah operasi `update` untuk memungkinkan `rollback` ke titik ini jika diperlukan di kemudian hari.

2. *Query*

```
INSERT INTO copy_departments(department_id,  
department_name, manager_id, location_id)  
VALUES(130, 'Estate Management', 102, 1500);  
  
UPDATE copy_departments SET department_id = 140;  
  
ROLLBACK TO SAVEPOINT one;  
  
COMMIT;
```

Hasil:

```
SQL> INSERT INTO copy_departments(department_id, department_name, manager_id  
, location_id) VALUES(130, 'Estate Management', 102, 1500);  
1 row created.  
SQL>  
SQL> UPDATE copy_departments SET department_id = 140;  
9 rows updated.  
SQL>  
SQL> ROLLBACK TO SAVEPOINT one;  
Rollback complete.  
SQL>  
SQL> COMMIT;  
Commit complete.
```

Gambar 12.2 Menampilkan bahwa data dengan department_id 130 telah dimasukkan, kemudian seluruh baris diubah menjadi department_id 140 namun dibatalkan dengan rollback ke savepoint one dan hasil akhir tidak berubah

Penjelasan:

Pada *query* tersebut digunakan untuk memasukkan data baru ke tabel copy_departments, lalu memperbarui semua department_id = 140. Namun, perubahan ini dibatalkan dengan ROLLBACK TO SAVEPOINT one yang mengembalikan kondisi tabel seperti saat savepoint one dibuat. COMMIT kemudian memastikan perubahan yang tersisa disimpan permanen.

3. *Query*

```
UPDATE wf_languages  
SET language_name = 'Bahasa Contoh'  
WHERE language_id = 104;  
  
ROLLBACK;
```

Hasil:

```
SQL> UPDATE wf_languages  
2 SET language_name = 'Bahasa Contoh'  
3 WHERE language_id = 104;  
  
1 row updated.  
  
SQL>  
SQL> ROLLBACK;  
  
Rollback complete.
```

Gambar 12.3 Menampilkan bahwa perubahan nilai `language_name` untuk `language_id` 104 dibatalkan oleh perintah `rollback`

Penjelasan:

Pada *query* tersebut digunakan untuk mengubah nama bahasa menjadi 'Bahasa Contoh' untuk baris dengan `language_id = 104`. Namun perubahan tersebut dibatalkan dengan perintah `ROLLBACK` sehingga data tetap seperti semula.

4. *Query*

```
INSERT INTO wf_currencies (currency_code,  
currency_name, comments)  
VALUES ('XYZ', 'Example Dollar', 'Contoh mata  
uang');  
  
COMMIT;
```

Hasil:

```
SQL> INSERT INTO wf_currencies (currency_code, currency_name, comments)  
2 VALUES ('XYZ', 'Example Dollar', 'Contoh mata uang');  
  
1 row created.  
  
SQL>  
SQL> COMMIT;  
  
Commit complete.
```

Gambar 12.4 Menampilkan bahwa data baru dengan `currency_code` XYZ berhasil ditambahkan ke tabel `wf_currencies`

Penjelasan:

Pada *query* tersebut digunakan untuk menambahkan data baru ke dalam tabel `wf_currencies`. Baris data yang dimasukkan berisi tiga nilai, yaitu 'XYZ' sebagai kode mata uang (`currency_code`), 'Example Dollar' sebagai nama mata uang (`currency_name`), dan 'Contoh mata uang' sebagai komentar atau keterangan tambahan (`comments`). Setelah perintah `INSERT` dijalankan, perintah `COMMIT` digunakan untuk menyimpan perubahan secara permanen ke dalam basis data.

5. *Query*

```
DELETE FROM wf_countries
WHERE country_id IN (2, 506);

SAVEPOINT sp_before_currency;

UPDATE wf_currencies
SET comments = 'Diperbaharui pada ' ||
TO_CHAR(SYSDATE, 'YYYY-MM-DD')
WHERE currency_code = 'USD';

ROLLBACK TO SAVEPOINT sp_before_currency;

COMMIT;
```

Hasil:

```
SQL> DELETE FROM wf_countries
  2  WHERE country_id IN (2, 506);

2 rows deleted.

SQL> SAVEPOINT sp_before_currency;

Savepoint created.

SQL> UPDATE wf_currencies
  2  SET comments = 'Diperbaharui pada ' || TO_CHAR(SYSDATE, 'YYYY-MM-DD')
  3  WHERE currency_code = 'USD';

1 row updated.

SQL> ROLLBACK TO SAVEPOINT sp_before_currency;

Rollback complete.

SQL> COMMIT;

Commit complete.
```

Gambar 12.5 Menampilkan bahwa dua baris pada `wf_countries` dihapus, kemudian komentar mata uang USD sempat diperbarui tetapi dibatalkan dan hanya penghapusan yang disimpan

Penjelasan:

Pada *query* tersebut digunakan untuk menghapus dua baris dari tabel `wf_countries` berdasarkan `country_id` 2 dan 506. Setelah itu, dibuat savepoint bernama `sp_before_currency` sebagai titik pemulihan. Selanjutnya, dilakukan update pada kolom `comments` di tabel `wf_currencies` untuk mata uang 'USD', dengan menambahkan

keterangan tanggal pembaruan. Namun, `update` tersebut dibatalkan menggunakan `ROLLBACK TO SAVEPOINT`, sehingga hanya perubahan setelah savepoint yang diurungkan. Penghapusan data tetap berlaku, dan perintah `COMMIT` digunakan untuk menyimpan perubahan tersebut secara permanen.

12.4 Latihan

1. Menggunakan perintah TCL untuk mengelola transaksi data dalam tabel.

Query

```
INSERT INTO mahasiswa (nim, nama, prodi)
VALUES ('2407123457', 'Zidan', 'TI');
SAVEPOINT simpan1;
INSERT INTO mahasiswa (nim, nama, prodi)
VALUES ('2407987653', 'Rifqi', 'TE');
SAVEPOINT simpan2;
INSERT INTO mahasiswa (nim, nama, prodi)
VALUES ('2407321097', 'Ayo', 'TE');
```

Hasil:

```
SQL> INSERT INTO mahasiswa (nim, nama, prodi) VALUES('2407123457',
'Zidan', 'TI');

1 row created.

SQL> SAVEPOINT simpan1;

Savepoint created.

SQL> INSERT INTO mahasiswa (nim, nama, prodi) VALUES ('2407987653',
'Rifqi', 'TE');

1 row created.

SQL> SAVEPOINT simpan2;

Savepoint created.

SQL> INSERT INTO mahasiswa (nim, nama, prodi) VALUES('2407321097',
'Ayo', 'TE');

1 row created.

SQL> ROLLBACK TO simpan2;

Rollback complete.

SQL>
SQL> COMMIT;

Commit complete.
```

Gambar 12.6 Menunjukkan bahwa data pegawai sempat dihapus tetapi dibatalkan, dan perubahan terakhir yang tersimpan adalah jabatan menjadi Senior Staff IT

Penjelasan:

Pada *query* tersebut digunakan untuk melakukan transaksi data secara bertahap ke dalam tabel mahasiswa dengan memanfaatkan perintah TCL untuk mengelola penyimpanan dan pembatalan data. Pertama, data mahasiswa dengan NIM '2307123457', nama "Zidan", dan prodi "TI" dimasukkan ke dalam tabel, lalu dibuat `SAVEPOINT` bernama `simpan1` sebagai penanda bahwa data ini telah berhasil disisipkan. Kemudian, data mahasiswa kedua ('2407987653', 'Rifqi', 'TE') ditambahkan dan disimpan dalam `SAVEPOINT` kedua (`simpan2`). Setelah itu, data ketiga ('2407321097', 'Ayos', 'TE') dimasukkan, namun karena dilakukan `ROLLBACK TO simpan2`, maka penyisipan data ketiga dibatalkan. Terakhir, perintah `COMMIT` digunakan untuk menyimpan perubahan yang telah dilakukan sebelumnya secara permanen ke dalam *database*. Dengan kata lain, hanya data Zidan dan Rifqi yang akan benar-benar disimpan, sedangkan data Ayos tidak disimpan karena telah dibatalkan sebelum `commit`.

12.5 Kesimpulan

Berdasarkan teori dan penerapan yang telah dilakukan dalam praktikum *Transaction Control Language* (TCL), dapat disimpulkan bahwa penggunaan perintah TCL sangat penting dalam mengelola transaksi data secara aman dan terkendali di dalam sistem basis data relasional. Praktikum ini memberikan pemahaman praktis tentang bagaimana perintah `COMMIT`, `ROLLBACK`, dan `SAVEPOINT` dapat digunakan untuk mengatur kapan data disimpan secara permanen, kapan perubahan dapat dibatalkan, serta bagaimana mengelola bagian tertentu dari transaksi melalui titik pemulihan.

Melalui percobaan-percobaan menggunakan perintah `INSERT`, `UPDATE`, dan `DELETE`, peserta praktikum dapat memahami bahwa perubahan data tidak langsung tersimpan secara permanen, melainkan harus dikendalikan dengan TCL agar terhindar dari kesalahan atau kehilangan data. Penggunaan `SAVEPOINT` juga terbukti berguna dalam transaksi yang kompleks, di mana sebagian data ingin dibatalkan tanpa mempengaruhi keseluruhan proses.

Dengan memahami konsep dan fungsi utama dari TCL, peserta praktikum dapat mengaplikasikan teknik pengelolaan transaksi secara lebih efektif dalam pengembangan sistem nyata, seperti pada sistem keuangan, akademik, maupun aplikasi yang melibatkan banyak pengguna dan data yang sensitif. Oleh karena itu, penguasaan TCL bukan hanya penting dari sisi teknis, namun juga merupakan keterampilan fundamental dalam memastikan integritas dan keandalan data dalam manajemen basis data.